

RANDOM-VALUED IMPULSE NOISE DETECTOR FOR SWITCHING MEDIAN FILTERS USING EDGE DETECTORS

Robert Grou-Szabo, Tadashi Shibata

Department of Frontier Informatics, Graduate School of Frontier Science,
The University of Tokyo, 7-3-1 Hongo, Bunkyo-ku, Tokyo, 113-8656, Japan
grou@if.t.u-tokyo.ac.jp - shibata@ee.t.u-tokyo.ac.jp

Abstract— This paper talks about a random impulse-noise detection scheme that was developed to be used as the first stage of a switching median filter. First by using an edge detector with a dynamic threshold, hidden artifacts are revealed by gradually changing the threshold of the edge detector. Then by applying a series of masks to the edge maps, the artifacts are pinpointed and tabulated. By analyzing the statistical data, the intensity of the random impulse noise corrupting a pixel is estimated. This information can then be passed on to an adaptive noise reduction filter that can eliminate it with a correlated intensity.

I. INTRODUCTION

Noise reduction is one of the most basic processes in image processing, yet it is considered to be one of the most important. It is essential to eliminate noise from an image before subsequent processing, such as edge detection, image segmentation or object recognition, can be done with high accuracy. The critical challenge is to preserve uncorrupted pixels while eliminating ‘noisy’ pixels. The two most common types of image-corrupting noises are white Gaussian noise and random impulse noise. This paper considers the latter type.

Random impulse noise, also known as spike noise or Salt and Pepper, occurs when a pixel takes on a completely random value regardless of the pixel’s original value. In most computer applications this ranges from 0 to 255. Impulse noise is due to various unpredictable causes such as spontaneous random spikes in either the power levels or in the light intensity when the image is being captured. It can also be the result of a light photon hitting the CCD at a different angle than its surrounding neighbors depending on the specular level or the reflectivity of the surface being photographed, or the result of one photodiode spontaneously not recording the same light intensity value as the neighboring photodiodes in the array. A similar phenomenon can be observed by looking at lens flares or when photographing in a very bright location where the photoresistors are being overloaded or in a low-light situation where not all transistors are triggered at the same time. Regardless of the reason, impulse noise is more widespread in digital photography than most people think. The reason it often goes unnoticed is that corrupted pixels are usually affected in only one color channel at a time. Consequently the other two color channels dampen the impact of the distortion caused by a pixel corrupted in only one

channel. For example a pixel might record the correct values for the Red and Green channels but a substantially incorrect value for the Blue channel; but because of the two out of three correct channels, the result is a pixel that appears only slightly different than its neighbors.

II. RELATED WORK

The simplest and most effective choice of noise filter to eliminate impulse noise is the Median Filter. However it has a few drawbacks. The algorithm involves sweeping a window across the entire image and can equally affect ‘good’ pixels as well as noisy pixels. Moreover the median filter has some difficulty dealing with thin lines. The centre weighted median filter (CWMF) was developed to improve the median filter [2], [8], [9]. But this filter is susceptible to the same problem of modifying every pixel regardless of whether it is corrupted or not.

To overcome these problems algorithms using a “surgical strike” approach, such as the switching median filter (SMF) [3]-[4] were developed. It consists of a two step algorithm. The first phase locates and targets corrupted pixels. In the second phase, pixels flagged as noisy are filtered. This paper deals with the first phase of the Switching Median Filter. Several strategies exist to locate and specifically target a pixel that has been corrupted by random impulse noise [5]-[7]. From the onset, the two constraints that had to be solved were first that truly random-valued impulse noise and not Salt & Pepper noise was to be eliminated and second that a “hardware-friendly” algorithm was to be produced so that it could easily be implemented in VLSI. Salt & Pepper noise occurs when a pixel simply takes on either the maximum or minimum value possible generating black or white pixels. Random-valued noise is far more likely in a real-world application which is why it was chosen.

Previously, a dynamic thresholding technique was used to detect JPEG block effects [1]. In this paper we push the technique further and adapt it to detect Random Impulse Noise as well.

III. PROPOSED ALGORITHM

The algorithm essentially consists of iteratively sweeping an edge detector over an image while gradually increasing the

threshold of the edge detector at each iteration pass. Accumulating the edge information for each threshold iteration value we build an edge histogram which is statistically analyzed. Using the statistical data we can detect random impulse noise and pass on this information to a second stage noise filter. The information transferred to the filter also contains a noise intensity flag indicating how corrupted a pixel is compared to its surrounding neighborhood and consequently filter the pixel more or less severely according to this flag value. We initially decided to use edge detectors to extract features from an image because edge information is crucial to the Human Visual System (HVS). Objects in an image are still easily recognizable when the color have been modified or if part of an object is hidden as long as enough of the object's outline remains allowing the brain to fill in the missing gaps and close the object's silhouette. This is why edge information is arguably the most important characteristic in an image for any animal's perceptual system to operate properly. We use edge-based statistical analysis as the starting direction for our research. Below is the step-by-step pseudocode followed by an outline of the random impulse noise detection algorithm:

```

Load TestImage
For Thresh = 0 to 255
    {
        Horiz_EdgeMap = Prewitt Edge (TestImage, Thresh, horizontal)
        Vert_EdgeMap = Prewitt Edge (TestImage, Thresh, vertical)

        OREdgeMap = Logical or (Horiz_EdgeMap, Vert_EdgeMap)

        Flagged_pixels = imfilter(OREdgeMap, "Masks from Fig. 2")
        Cumulated_flags = (Cumulated_flags, Flagged_pixels)

    }
Remove all flagged pixels from Cumulated_flags that are smaller than the LTL

```

Step 1) First an edge detector is iteratively applied to the image as the sensitivity's threshold value is increased from 0 to 255 at each iteration pass. We've experimented using several kinds of edge detectors and found that directional edge detectors work best so in our experiments we used the Prewitt edge detector.

Step 2) Then, for each iteration phase, first the edge detector is passed in a horizontal direction to produce a horizontal edge map Fig. 1-A. The same is done in a vertical direction to produce a vertical edge map, Fig. 1-B, using the same threshold value as the horizontal edge map.

Step 3) Once both edge maps have been generated, a logical 'OR' is computed from both edge maps to produce an 'OREdgeMap' illustrated in Fig. 1-C. By observing the image closely, some distinctive features can be noticed. Peculiar 'doughnut' shaped squares appear at the intersection of the edge map where impulse noise is corrupting a pixel. This is done for each threshold value from 0 to 255.

Step 4) Then a series of masks (illustrated in Fig. 2) are passed over the OREdgeMap, thereby locating the 'doughnut' artifacts that actually correspond to pixels that have been corrupted by impulse noise. When a pixel has been highly corrupted by impulse noise and is very distinct from its surrounding pixels, it will result in a nice square-shaped artifact. On the other hand if the noisy pixel's value is not so different from its neighbors, and especially when it is in an area with a color gradient, then different kinds of artifacts are produced which explains the need for a variety of mask shapes.

Step 5) During the mask filtering process, if a pixel is found to be at the centre of one of those doughnut masks, that pixel is flagged and the filtering process continues for that OREdgeMap. This is repeated for each OREdgeMap at every iterative threshold value. If a corrupted pixel continues to engender toroidal-shaped artifacts around it at each iteration, then that pixel is flagged once each iteration. The more disparity there is between a pixel and its neighbors, the more often it will get tagged as being corrupted and subsequently will need to be filtered more severely. However if a 'doughnut' artifact appears for only a few iterations and is tagged only once or twice, then that pixel may not need to be filtered at all. Fig. 3 shows a test image flat in the horizontal plane, and in the Z-axis are the stems where pixels have been flagged during the mask filtering phase. The height of each stem corresponds to how many times a pixel has been flagged. These are the cumulative flags for every OREdgeMap spanning each iteration at every threshold value. The higher the stem, the more corrupt a pixel is and therefore must be

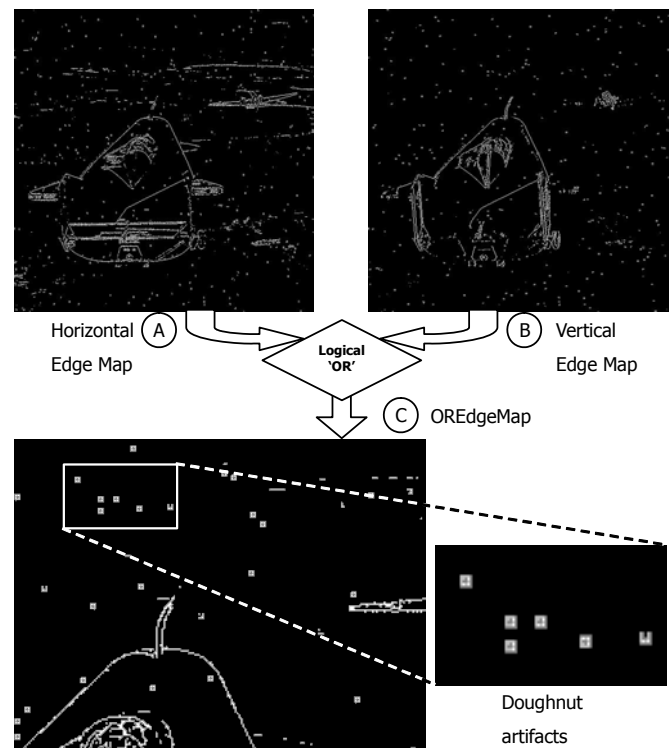


Fig. 1. A) Horizontal edge map; B) Vertical edge map; C) Logical 'OR' using Horizontal and Vertical edge maps as inputs depicts 'doughnut' shapes where random impulse noise is present

filtered more strongly. Shorter stems simply means that an artifact was detected at that pixel's location but that it immediately disappeared with subsequent iteration passes. In that case a pixel with a corresponding stem that has a length of only 1 means that although some particularity exists at that pixel, it's value is very close to the surrounding neighbors and therefore any noise filtering that might occur could make things worst than they already are.

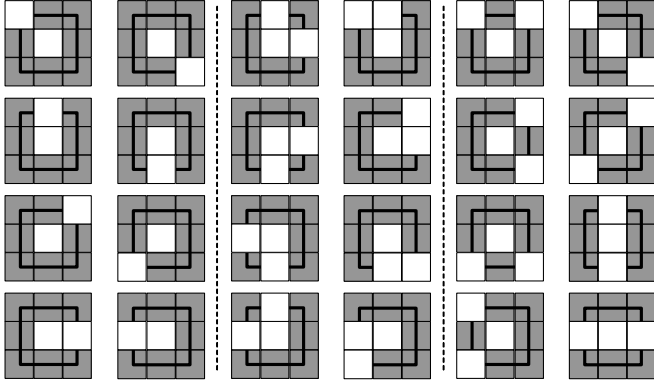


Fig. 2. 3x3 pixel masks used to detect 'doughnut' artefacts from the OREdgeMap. Grey-coloured squares are ones and white squares, zeros.

Step 6) Once a stem map (fig. 3) of all flagged pixels has been generated a "Lower Threshold Limit" value (LTL) must be chosen. This is the minimum number of times a pixel needs to be flagged to be considered as noisy. Stems that are longer than the LTL are filtered whereas stems that are shorter are ignored.

Figure 4 is an OREdgeMap at a very low threshold value and as a result many artifacts appear, some of which look like the 'doughnut' masks of Figure 2. However these artifacts are not caused by actual impulse noise, rather they are the result of using a very sensitive edge detector. After sweeping all of the masks over Fig. 4, many pixels would be flagged as containing noise where none exists which is why the Low Threshold Limit value is needed to weed out pixels that were flagged only a few times. The LTL is a parameter selected by the user, but the following two factors should be taken into

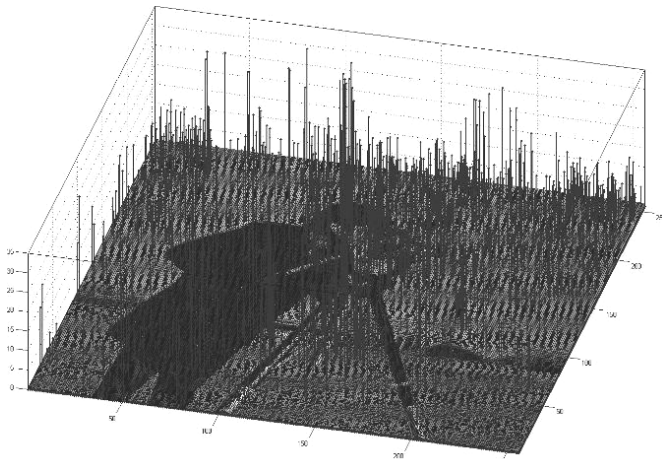


Fig. 4. Accumulative 'Stem-view' of pixels that have been flagged as being corrupt whenever a 'donut' artefact was detected by the masks

account. First is the dimension of the image; second is the amount of edge information contained in the image itself. Larger images, say greater than 128x128 pixels, are less impacted by the Lower Threshold Limit, but as the test image gets smaller (for example a 32x32 Desktop icon), there is much less feature information and each pixel becomes more important. It also becomes more difficult to determine if a pixel is noisy or if it simply represents a large jump in pixel color. The second factor that can have an impact on the LTL is the amount of "Edge information" in an image. In other words, a smooth image of a calm sea will have very little edge information in it and therefore it will be very easy to bring out any peculiar artifacts. However an image containing a crowd of faces with a lot of edge information and that has a lot of high frequencies will be much more difficult to differentiate any noise from all the background noise. So to obtain the best results, an adaptive LTL can be used, whose value changes with the volume of edges detected in the OREdgeMap.

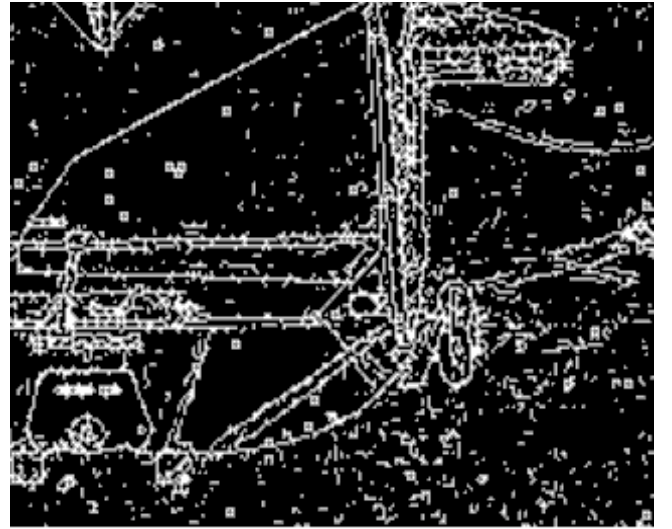


Fig. 3. OREdgeMap at a low threshold value, where many artefacts that are NOT due to impulse noise can be observed

IV. RESULTS

Simulations were carried out using test images from Matlab. The white dots in Fig. 5-Left are the pixels where impulse noise was artificially added while the circles shown in 5-Right shows where the impulse noise was detected by the algorithm. The dots with no circles around them in Fig. 5-Right are noise that have not been detected and circles with no dots at the centre are pixels that have been falsely identified as being noisy. It is worth noting that the white dots in both sides of Figure 5 are random valued which means that even those pixels that were corrupted by noise but weren't detected could mean that even though the pixel's value was randomly distorted, the new pixel value was close enough to its original uncorrupted value and in this case it would be ignored because the difference between the pixel and its neighbors is smaller than the LTL value.

In our simulations, the Lower Threshold Limit with the best impulse noise detection ratio was between 10 and 13. This means that for a pixel to be identified as noise, it has to be different from its neighbors by about 10 color gradients. If

the LTL is too low, then a lot of background noise is falsely identified as impulse noise and the LTL limit is raised, at some point the amount of background noise falls off and only the impulse noise is left. Then as the LTL passes 15 color gradients, so then does the sensitivity decrease and as a result there are more corrupted pixels that go undetected but with fewer false positives.

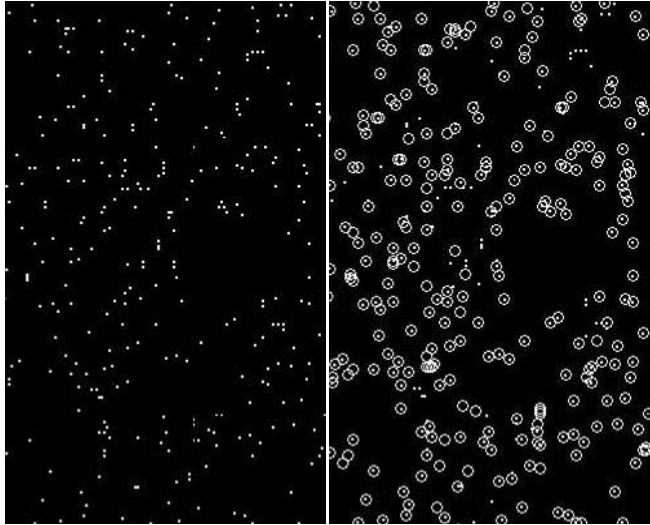


Fig. 5. *Left side:* The white specs represent random impulse noise added to an image. *Right side:* The same image processed with the proposed algorithm. The circles are pixels where the algorithm detected impulse noise. Circles with no dots at the centre are 'false positives' and dots without circles are undetected noise.

V. CONCLUSION

In this paper we proposed a novel random impulse-noise detection scheme for use as the first stage of a switching median filter (SMF). Our aim was to develop a "hardware-friendly" algorithm that could easily be implemented in a VLSI design. The proposed algorithm is thought to be hardware friendly because the essential component is essentially an edge detector, as opposed to other algorithms that use complex mathematical tools such as fuzzy relations or mathematical morphology. Most of the operations needed for this algorithm can easily be implemented into a pipelined

architecture capable of processing streaming video. An added advantage of this algorithm is that a sort of 'fuzzy flag' is also generated for each pixel that has been deemed corrupt so that the subsequent filtering step can be applied with more or less intensity and this is all done without any previous training of any kind. A known problem with this algorithm is that if 2 adjacent pixels become corrupted, then the masks will not work very well and neither pixel would be identified as being corrupted. Also, if the corrupted pixel is on an edge and the "random value" that it acquires is actually close to one or either sides of the boundary, then detection also become difficult.

REFERENCES

- [1] R. Grou-Szabo and T. Shibata, "Blind image compression history determination using dynamic thresholding," in Proc. ICASSP 2008, pp. 1005-1008, April 2008.
- [2] S.J. Ko and Y. H. Lee, "Center weighted median filters and their applications to image enhancement", IEEE Trans. Circuits Syst. Vol. 38, NO. 9, pp. 984-993, Sept. 1991
- [3] Zhou Wang, David Zhang "Progressive Switching Median Filter for the Removal of Impulse Noise from Highly Corrupted Image", IEEE Trans. Circuits & Systems II: Analog & Digital Signal Processing, Vol. 46, NO. 1, pp. 78-80, Jan 1999
- [4] Shuqun Zhang and Mohammad A. Karim, "A New Impulse Detector for Switching Median Filters" IEEE Signal Processing Letters Vol. 9, NO. 11, November 2002, pp. 360-363
- [5] Roman Garnett, Timothy Huegerich, Charles Chui, Wenjie He "A Universal Noise Removal Algorithm with an Impulse Detector", IEEE Transactions on Image Processing Vol. 14, NO. 11, November 2005, pp. 1747-1754
- [6] Schulte S., De Witte V., Nachtegaal M., Van der Weken D., Kerre E.E., "Fuzzy two-step filter for impulse noise reduction from color images", IEEE transactions on image processing, Vol. 15 NO. 11, pp. 3568-3579, Nov. 2006
- [7] Yamashita N., Munenori O., Jianming L., Hiroo S., Takashi Y., "A Random-valued Impulse Noise Detector Using Level Detection" Circuits and Systems, 2005. ISCAS 2005. IEEE International Symposium on, May 2005, pp. 6292-6295
- [8] Chen T., Hong Ren Wu, "Adaptive impulse detection using center-weighted median filters," Signal processing Letters, IEEE, Vol. 8, No. 1, Jan. 2001, pp. 1-3
- [9] D. R. K. Brownrigg, "The weighted median filter," Commun. ACM, Vol. 27, No. 8, Aug. 1984, pp. 807-818